

Research Notes on StyleGAN

StyleGAN

>> Section 1

StyleGAN is designed as a combination of Progressive GAN with neural style transfer. The key architectural choice of StyleGAN-1 is a progressive growth mechanism, similar to Progressive GAN. Each generated image starts as a constant $4 \times 4 \times 512$ array, and repeatedly passed through style blocks. Each style block applies a "style latent vector" via affine transform ("adaptive instance normalization"), similar to how neural style transfer uses Gramian matrix. It then adds noise, and normalize (subtract the mean, then divide by the variance). At training time, usually only one style latent vector is used per image generated, but sometimes two ("mixing regularization") in order to encourage each style block to independently perform its stylization without expecting help from other style blocks (since they might receive an entirely different style latent vector). After training, multiple style latent vectors can be fed into each style block. Those fed to the lower layers control the large-scale styles, and those fed to the higher layers control the fine-detail styles. Style-mixing between two images x, x' can be performed as well. First, run a gradient descent to find z, z' such that $G(z) \approx x, G(z') \approx x'$. This is called "projecting an image back to style latent space". Then, z can be fed to the lower style blocks, and z' to the higher style blocks, to generate a composite image that has the large-scale style of x , and the fine-detail style of x' . Multiple images can also be composed this way.

>> Section 2

StyleGAN2 StyleGAN2 improves upon StyleGAN in two ways. One, it applies the style latent vector to transform the convolution layer's weights instead, thus solving the "blob" problem. The "blob" problem roughly speaking is because using the style latent vector to normalize the generated image destroys useful information. Consequently, the generator learned to create a "distraction" by a large blob, which absorbs most of the effect of normalization (somewhat similar to using flares to distract a heat-seeking missile). Two, it uses residual connections, which helps it avoid the phenomenon where certain features are stuck at intervals of pixels. For example, the seam between two teeth may be stuck at pixels divisible by 32, because the generator learned to generate teeth during stage $N-5$, and consequently could only generate primitive teeth at that stage, before scaling up 5 times (thus intervals of 32). This was updated by the StyleGAN2-ADA ("ADA" stands for "adaptive"), which uses invertible data augmentation. It also tunes the amount of

data augmentation applied by starting at zero, and gradually increasing it until an "overfitting heuristic" reaches a target level, thus the name "adaptive".

>> Section 3

StyleGAN3 StyleGAN3 improves upon StyleGAN2 by solving the "texture sticking" problem, which can be seen in the official videos. They analyzed the problem by the Nyquist–Shannon sampling theorem, and argued that the layers in the generator learned to exploit the high-frequency signal in the pixels they operate upon. To solve this, they proposed imposing strict lowpass filters between each generator's layers, so that the generator is forced to operate on the pixels in a way faithful to the continuous signals they represent, rather than operate on them as merely discrete signals. They further imposed rotational and translational invariance by using more signal filters. The resulting StyleGAN-3 is able to generate images that rotate and translate smoothly, and without texture sticking.

>> Section 4

Illicit use In December 2019, Facebook took down a network of accounts with false identities, and mentioned that some of them had used profile pictures created with machine learning techniques.

>> Section 5

Progressive GAN Progressive GAN is a method for training GAN for large-scale image generation stably, by growing a GAN generator from small to large scale in a pyramidal fashion. Like SinGAN, it decomposes the generator as $G = G_1 \circ G_2 \circ \dots \circ G_N$, and the discriminator as $D = D_N \circ D_{N-1} \circ \dots \circ D_1$. During training, at first only G_N, D_N are used in a GAN game to generate 4×4 images. Then G_{N-1}, D_{N-1} are added to reach the second stage of GAN game, to generate 8×8 images, and so on, until we reach a GAN game to generate 1024×1024 images. To avoid discontinuity between stages of the GAN game, each new layer is "blended in" (Figure 2 of the paper). For example, this is how the second stage GAN game starts:

>> Section 6

Just before, the GAN game consists of the pair G_N, D_N generating and discriminating 4×4 images. Just after, the GAN game consists of the pair $((1-\alpha) + \alpha \cdot G_{N-1}) \circ G_N, D_N \circ ((1-\alpha) + \alpha \cdot D_{N-1})$

Research Notes on StyleGAN

StyleGAN

$\alpha \cdot D_{N-1}$ generating and discriminating 8×8 images. Here, the functions u, d are image up- and down-sampling functions, and α is a blend-in factor (much like an alpha in image compositing) that smoothly glides from 0 to 1.

>> Section 7

The Style Generative Adversarial Network, or StyleGAN for short, is an extension to the GAN architecture introduced by Nvidia researchers in December 2018, and made source available in February 2019. StyleGAN depends on Nvidia's CUDA software, GPUs, and Google's TensorFlow, or Meta AI's PyTorch, which supersedes TensorFlow as the official implementation library in later StyleGAN versions. The second version of StyleGAN, called StyleGAN2, was published on February 5, 2020. It removes some of the characteristic artifacts and improves the image quality. Nvidia introduced StyleGAN3, described as an "alias-free" version, on June 23, 2021, and made source available on October 12, 2021.